

TUGAS 3
MATA KULIAH
Struktur Data / STSI4104.104



UNIVERSITAS TERBUKA

DISUSUN OLEH

NAMA : Yudo Sandewo
NIM : 052968524
KODE KELAS : 104

PROGRAM STUDI ILMU SISTEM INFORMASI

Fakultas Sains dan Teknologi

UPBJJ UT JAKARTA

UNIVERSITAS TERBUKA

Nama: Yudosandewo

NIM: 052968524

Prodi: Sistem Informasi

Source Code :

```
import java.util.*;

public class tugas3 {

    /*
     * =====
     * CLASS NODE
     * - Mewakili 1 node pada graf
     * - Menyimpan label ("a1", "a2", ...) dan nilai integer
     * =====
     */
    static class Node {
        String label;
        int value;

        Node(String label, int value) {
            this.label = label;
            this.value = value;
        }

        @Override
        public String toString() {
            return label + "(" + value + ")";
        }
    }

    /*
     * =====
     * BAGIAN ATRIBUT GRAF
     * nodes = menyimpan 8 node a1..a8 beserta nilainya
     * adj = adjacency list (hubungan antar node)
     * =====
     */
    private Node[] nodes;
    private List<Integer>[] adj;

    @SuppressWarnings("unchecked")
    public tugas3() {

        /*
         * =====
         * INISIALISASI NODE (a1..a8)
         * - Kita buat array berisi 8 node
         * - Masing-masing punya value berbeda
         * =====
         */
        nodes = new Node[8];
        nodes[0] = new Node("a1", 3);
        nodes[1] = new Node("a2", 5);
        nodes[2] = new Node("a3", 7);
        nodes[3] = new Node("a4", 2);
```

```

        nodes[4] = new Node("a5", 9);
        nodes[5] = new Node("a6", 1);
        nodes[6] = new Node("a7", 8);
        nodes[7] = new Node("a8", 6);

        /*
         * =====
         * INISIALISASI ADJACENCY LIST
         * - Setiap node akan punya list tetangga
         * =====
         */
        adj = new ArrayList[8];
        for (int i = 0; i < 8; i++)
            adj[i] = new ArrayList<>();

        /*
         * =====
         * DEFINISI EDGE (GRAF TAK-BERARAH)
         * - Kita sambungkan node-node sesuai struktur graf
         * - addEdge(u,v) otomatis menambah 2 arah
         * =====
         */
        addEdge(0, 1); // a1 - a2
        addEdge(0, 2); // a1 - a3
        addEdge(1, 3); // a2 - a4
        addEdge(1, 4); // a2 - a5
        addEdge(2, 5); // a3 - a6
        addEdge(3, 6); // a4 - a7
        addEdge(4, 6); // a5 - a7
        addEdge(4, 7); // a5 - a8
        addEdge(5, 7); // a6 - a8
    }

    /*
     * =====
     * addEdge: Fungsi menambahkan edge tak-berarah
     * - u terhubung ke v
     * - v terhubung ke u
     * =====
     */
    private void addEdge(int u, int v) {
        adj[u].add(v);
        adj[v].add(u);
    }

    /*
     * =====
     * DFS (Depth-First Search)
     * - Menggunakan rekursi
     * - Menyelam sedalam mungkin sebelum mundur
     * - visitOrder mencatat urutan kunjungan
     * =====
     */
    public String dfs(int startIndex, int n) {
        boolean[] visited = new boolean[nodes.length]; // tanda sudah dikunjungi

```

```

        List<String> visitOrder = new ArrayList<>(); // menyimpan urutan traversal

        String result = dfsRec(startIndex, n, visited, visitOrder);

        System.out.println("DFS visit order: " + visitOrder);
        return result;
    }

    /*
    * =====
    * dfsRec: fungsi rekursif utama DFS
    *
    * Tahapan:
    * 1. Tandai node saat ini sebagai sudah dikunjungi
    * 2. Tambahkan ke list visitOrder
    * 3. Jika nilainya cocok → return label node
    * 4. Jika tidak:
    * - Telusuri semua tetangga yang belum dikunjungi
    * - Jika ditemukan di cabang, bubble up hasilnya
    * =====
    */
    private String dfsRec(int current, int n, boolean[] visited, List<String>
visitOrder) {
        visited[current] = true; // (1) tandai
        visitOrder.add(nodes[current].toString()); // (2) simpan urutan kunjungan

        if (nodes[current].value == n) { // (3) apakah ini node yang dicari?
            return nodes[current].label;
        }

        // (4) telusuri tetangga
        for (int neighbor : adj[current]) {
            if (!visited[neighbor]) {
                String found = dfsRec(neighbor, n, visited, visitOrder);
                if (found != null) // kalau ditemukan, naikan hasilnya
                    return found;
            }
        }

        return null; // tidak ditemukan di cabang ini
    }

    /*
    * =====
    * BFS (Breadth-First Search)
    * - Menggunakan queue
    * - Mengunjungi node berdasarkan level (melebar dulu)
    * - Iteratif (tidak rekursif)
    * =====
    */
    public String bfs(int startIndex, int n) {
        boolean[] visited = new boolean[nodes.length];
        Queue<Integer> q = new LinkedList<>();
        List<String> visitOrder = new ArrayList<>();
    }

```

```

        visited[startIndex] = true;
        q.add(startIndex); // masukkan node awal

        while (!q.isEmpty()) {
            int cur = q.poll(); // ambil dari queue
            visitOrder.add(nodes[cur].toString());

            // Apakah node ditemukan?
            if (nodes[cur].value == n) {
                System.out.println("BFS visit order: " + visitOrder);
                return nodes[cur].label;
            }

            // Tambahkan seluruh tetangga
            for (int neighbor : adj[cur]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.add(neighbor);
                }
            }
        }

        System.out.println("BFS visit order: " + visitOrder);
        return null;
    }

    /*
    * =====
    * MAIN PROGRAM
    * - Menampilkan graf
    * - Input nilai n
    * - Menjalankan DFS dan BFS
    * =====
    */
    public static void main(String[] args) {
        tugas3 g = new tugas3();
        Scanner sc = new Scanner(System.in);

        System.out.println("Graf terdiri dari node: ");
        for (Node node : g.nodes)
            System.out.println(" - " + node);

        System.out.print("\nMasukkan nilai n yang ingin dicari: ");
        int n = sc.nextInt();

        System.out.println("\n=== Depth-First Search (DFS) mulai dari a1 ===");
        String foundDfs = g.dfs(0, n);
        if (foundDfs != null)
            System.out.println("DFS: nilai " + n + " ditemukan pada node " + foundDfs
+ ".");
        else
            System.out.println("DFS: nilai " + n + " TIDAK ditemukan.");

        System.out.println("\n=== Breadth-First Search (BFS) mulai dari a1 ===");
        String foundBfs = g.bfs(0, n);
    }

```

```
        if (foundBfs != null)
            System.out.println("BFS: nilai " + n + " ditemukan pada node " + foundBfs
+ ".");
        else
            System.out.println("BFS: nilai " + n + " TIDAK ditemukan.");

        sc.close();
    }
}
```

Tautan video:

<https://youtu.be/VE35fgvI7oA?si=pJ2n-OQPttP1v73O>